CS2008 Numerical Computing — Assignment 2

Interpolation & Fitting (Code + Hand Questions)

Due: **15 Nov 2025** Submit a single PDF + a .zip of your code

Overview

This assignment blends coding and theory. You will implement classical interpolation methods and simple model fitting in Python, then answer two short hand questions.

What to Submit

- **PDF**: written answers and small result tables.
- **Code ZIP**: your completed `interpolation.py` (starter below) + any small plots you generated.

Grading (100 pts)

Part A: Lagrange (code + test)	15
Part B: Newton divided differences (code + test)	15
Part C: Newton forward/backward (code + test)	15
Part D: Linear & natural cubic spline (code + test)	20
Part E: Least squares polynomial fit (code + test)	15
Part F: Non-polynomial exponential fit (code + test)	10
Part G: Two hand questions (write-up)	10

Dataset for All Parts

Use the smooth test function:

$$f(x) = e^{-x/3}\big(\sin(2x) + 1.5\big), \quad x \in [0, 4].$$

Unless a part says otherwise, generate $n = 9$ *equally spaced* nodes on $[0, 4]$ and evaluate error on a dense grid of 400 points in $[0, 4]$. Report MAE (mean absolute error) for each method.

Starter Code

Create a file named `interpolation.py` and start from this minimal scaffold:

Listing 1: interpolation.py starter (fill all TODOs)

```python
import numpy as np
from math import factorial

# ---------- common helpers ----------
```

```python
def f(x):
    return np.exp(-x/3.0) * (np.sin(2*x) + 1.5)

def make_nodes(n=9, a=0.0, b=4.0):
    x = np.linspace(a, b, n)
    y = f(x)
    return x, y

def mae(y_true, y_pred):
    return float(np.mean(np.abs(y_true - y_pred)))

# ---------- Part A: Lagrange ----------
def lagrange_eval(xn, yn, xq):
    """
    Evaluate Lagrange interpolating polynomial at points xq.
    TODO: implement direct Lagrange basis (O(n^2) per query).
    """
    xn = np.asarray(xn, float); yn = np.asarray(yn, float)
    xq = np.atleast_1d(xq).astype(float)
    n = len(xn)
    yq = np.zeros_like(xq)
    # TODO
    return yq if yq.size > 1 else yq.item()

# ---------- Part B: Newton (divided differences) ----------
def newton_divdiff_coeffs(xn, yn):
    """
    Return coefficients c0..c_{n-1} of Newton form using divided differences.
    TODO: build the divided-difference table in-place.
    """
    xn = np.asarray(xn, float); yn = np.asarray(yn, float)
    coef = yn.astype(float).copy()
    # TODO
    return coef

def newton_eval(xn, coef, xq):
    """
    Horner-like evaluation of Newton polynomial.
    TODO: evaluate using nested multiplication (reverse loop).
    """
    xn = np.asarray(xn, float)
    xq = np.atleast_1d(xq).astype(float)
    yq = np.zeros_like(xq)
    # TODO
    return yq if yq.size > 1 else yq.item()

# ---------- Part C: Newton forward/backward (equal spacing) ----------
def forward_diff_table(y):
    """
    Return list [^0 y, ^1 y, ..., ^{n-1} y]
    TODO: iteratively compute np.diff of the previous column.
    """
    y = np.asarray(y, float).copy()
    table = [y]
    # TODO
    return table

def newton_forward_eval(x0, h, table, xq):
```

```
    """
    Forward formula with u=(x-x0)/h.
    TODO: implement rising product and factorial scaling.
    """
    xq = np.atleast_1d(xq).astype(float)
    yq = np.zeros_like(xq)
    # TODO
    return yq if yq.size > 1 else yq.item()

def newton_backward_eval(xn, h, table, xq):
    """
    Backward formula with u=(x-x_n)/h.
    TODO: implement falling product and factorial scaling (using last entries).
    """
    xq = np.atleast_1d(xq).astype(float)
    yq = np.zeros_like(xq)
    # TODO
    return yq if yq.size > 1 else yq.item()

# ---------- Part D: Linear & Natural Cubic Splines ----------
def linear_spline_eval(xn, yn, xq):
    """
    Piecewise linear interpolation.
    TODO: locate interval via searchsorted, linearly blend endpoints.
    """
    xn = np.asarray(xn, float); yn = np.asarray(yn, float)
    xq = np.atleast_1d(xq).astype(float)
    # TODO
    return np.zeros_like(xq) if xq.size > 1 else 0.0

def natural_cubic_spline_build(xn, yn):
    """
    Build natural cubic spline second derivatives M.
    TODO: set up and solve tridiagonal system for M with natural BCs.
    Return (xn, yn, M).
    """
    xn = np.asarray(xn, float); yn = np.asarray(yn, float)
    n = len(xn)
    # TODO
    M = np.zeros(n)
    return (xn, yn, M)

def natural_cubic_spline_eval(spline, xq):
    """
    Evaluate natural cubic spline using Hermite-like segment formula.
    TODO: find segment, compute a,b and the cubic combination with M.
    """
    xn, yn, M = spline
    xq = np.atleast_1d(xq).astype(float)
    # TODO
    return np.zeros_like(xq) if xq.size > 1 else 0.0

# ---------- Part E: Least-squares polynomial fit ----------
def design_matrix_poly(x, degree):
    """
    Vandermonde-like matrix [1, x, x^2, ...].
    TODO: build columns x**k for k=0..degree (stacked as columns).
    """
```

```
    x = np.asarray(x, float)
    # TODO
    return np.zeros((x.size, degree+1))

def poly_least_squares_fit(x, y, degree):
    """
    Solve min ||Xb - y|| using np.linalg.lstsq.
    TODO: build X and solve for b.
    """
    X = design_matrix_poly(x, degree)
    # TODO
    b = np.zeros(degree+1)
    return b

def poly_eval(b, xq):
    """
    Evaluate polynomial with coefficients b (ascending powers).
    TODO: straightforward sum b_k * x^k.
    """
    xq = np.asarray(xq, float)
    # TODO
    return np.zeros_like(xq)

# ---------- Part F: Non-polynomial exponential fit ----------
def fit_exponential(x, y):
    """
    y  A * exp(Bx), y>0. Linearize by ln y = ln A + B x and least-squares.
    TODO: mask y>0, fit line to (x, ln y) and return A,B.
    """
    x = np.asarray(x, float); y = np.asarray(y, float)
    # TODO
    A, B = 1.0, 0.0
    return A, B

def eval_exponential(A, B, xq):
    return A * np.exp(B * np.asarray(xq, float))

# ---------- small runner for your checks ----------
if __name__ == "__main__":
    x, y = make_nodes(n=9, a=0.0, b=4.0)
    xq = np.linspace(0, 4, 400); ytrue = f(xq)
    # Fill methods as you implement and print MAE for sanity checks.
    # Example:
    # c = newton_divdiff_coeffs(x, y); yN = newton_eval(x, c, xq)
    # print("Newton MAE:", mae(ytrue, yN))
```

Tasks

For each part below: (i) complete the TODOs, (ii) test on $x \in [0, 4]$, $n = 9$, (iii) print MAE, and (iv) add a 1–2 sentence observation in your PDF.

Part A (15 pts): Lagrange

Implement `lagrange_eval`. Test: report MAE on the dense grid.

Part B (15 pts): Newton (divided differences)

Implement `newton_divdiff_coeffs` and `newton_eval`. Test: MAE.

Part C (15 pts): Newton forward/backward (equal spacing)

Implement forward/backward difference tables and evaluators. Verify h is constant. Compare MAE of forward vs backward on the same grid.

Part D (20 pts): Linear and Natural Cubic Splines

Implement `linear_spline_eval`, `natural_cubic_spline_build`, `natural_cubic_spline_eval`. Report MAE of both; comment briefly on smoothness.

Part E (15 pts): Least-squares polynomial fit

Fit degree-5 polynomial to (x, y) with `poly_least_squares_fit`. Report MAE; compare to cubic spline.

Part F (10 pts): Non-polynomial fitting (exponential)

Fit $y \approx Ae^{Bx}$ via linearization where $y > 0$. Report (A, B) and MAE. Briefly note why this model may or may not suit $f(x)$.

Part G (10 pts): solve by hand, show steps

G.1 **Derive** the Newton divided differences for $n = 3$ nodes x_0, x_1, x_2 and show that the Newton form equals the unique quadratic through (x_i, y_i). Start from

$$P_2(x) = c_0 + c_1(x - x_0) + c_2(x - x_0)(x - x_1)$$

and express c_0, c_1, c_2 in terms of $y[x_0]$, $y[x_0, x_1]$, $y[x_0, x_1, x_2]$. *Compute the explicit formulae* (no numbers needed).

G.2 For equally spaced nodes $x_i = x_0 + ih$, **prove** that the Newton forward polynomial with $u = (x - x_0)/h$ reproduces any polynomial of degree $\leq k$ exactly when truncated at order k. Use properties of forward differences of polynomials.

Presentation & Notes

- Put your MAE results in a small table in the PDF. Optional: add plots.
- If you use randomness anywhere, set `np.random.seed(42)`.
- Comment your code. Partial credit is awarded for clear, testable implementations.